SECURITY

Mei Nagappan

UNIVERSITY OF WATERLOO | FACULTY OF MATHEMATICS

Guideline 1: Enumerate Candidate Threat Models Before Writing Code

Before generating security-sensitive code, prompt the AI to enumerate threats that apply to your system, with CWE IDs and reasoning for each.

Treat the output as a candidate list to review and confirm , not a final authority.

Carry the confirmed threat list into all subsequent code generation prompts.

Guideline 1: Why This Matters

- LLMs fail most on context-dependent vulnerabilities like broken access control: these threats are invisible from code alone
- Threat identification must happen before code generation, not after
- Prompts that name CWE categories produce significantly fewer vulnerabilities
- OWASP and NIST both treat threat identification as a required upstream step
- An AI-generated candidate list mirrors a structured threat modeling session; then a human validates before acting

Guideline 1: Prompting for Threat Enumeration

- Describe the system in plain language: what it does, what data it handles, who calls it, and where it runs
- Request a table with three columns: Threat, CWE ID, and Reason it applies to this specific system
- Ask the model to use STRIDE categories (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege)
- Ask for only threats that are plausible given the described architecture - not a generic checklist

Guideline 1: Reviewing Enumerated Threats

- Treat the output as a candidate list: remove threats that do not apply to your deployment context
- Verify each CWE ID at cwe.mitre.org; hallucinated CWE numbers do occur
- Add threats you know from your own knowledge of the system that the model omitted
- Use the confirmed list as input to Guideline 2: paste it into all code generation prompts

Guideline 2: Declare the Threat Model in Every Security-Sensitive Prompt

When generating security-sensitive code, name the specific vulnerabilities to defend against and the exact libraries or standards to use.

A prompt that omits these constraints produces functionally correct code that silently skips defenses , even from a capable model.

Use the confirmed threat list from Guideline 1 as direct input to this step.

Guideline 2: Why This Matters

- Prompts naming vulnerability types produce significantly fewer CWE-classified weaknesses
- Study covered 150 prompts mapped to 18 OWASP Top 25 CWEs naming defenses matters
- AI models optimize for functional correctness given the information in the prompt
- Without an explicit security contract, the model has no signal to include defenses

Guideline 2: Prompting

- Paste the confirmed threat list from Guideline 1 directly as a numbered requirements section
- Name every vulnerability class by CWE: 'protect against path traversal (CWE-22)'
- Specify the exact library or mechanism: 'use bcrypt for password hashing', 'use parameterized queries'
- Include the full security contract in one prompt; do not ask for security as a follow-up
- Ask for error handling that does not leak internal state (no full paths in error messages)

Guideline 2: Reviewing the Result

- Check that every named defense from the threat list actually appears in the output
- Reject output that uses string interpolation in any query, command, or file path construction
- Verify error messages are generic and do not expose server internals

Guideline 3: Generate Exploit-Style Tests from the Threat Model

After generating hardened code, ask the AI to generate tests where each test targets a specific CWE from the threat model.

Each test has two parts: an exploit attempt that fails on a vulnerable implementation, and an assertion that confirms the defense holds.

This turns the threat model into a verifiable, re-runnable contract.

Guideline 3: Why This Matters

- LLMs cannot reliably reason about context-dependent vulnerabilities without explicit application context
- Tests written without the threat model will not target the right vulnerability class for the specific component
- Iterative AI repair is most effective when anchored to a failing test from a known vulnerability
- Unanchored iteration ('make it more secure') increases critical vulnerabilities by 37.6% after five rounds
- Exploit-style tests provide a binary, re-runnable verdict for each identified threat

Guideline 3: Prompting for Exploit-Style Tests

- Paste the confirmed threat model table from Guideline 1 alongside the code under test
- Ask for one test function per CWE entry, labelled with the CWE ID in the function name (e.g., test_cwe22_path_traversal)
- For each test: (a) describe the exploit attempt in a comment, (b) assert specific enough to fail on naive code and pass on hardened code
- Ask for property-based tests using Hypothesis for any CWE involving free-form string input, to cover attack variants
- Ask for a separate section of happy-path tests so security tests are clearly distinguished

Guideline 3: Reviewing Generated Tests

- Run the test suite against both the naive implementation and the hardened one: security tests must fail on the former and pass on the latter
- Check that assertions go beyond a non-500 status code: assert absence of sensitive data, correct status codes (403, 401), or rejected malicious input
- Verify that each test covers the CWE it claims to target, not a superficially related weakness

References

[1] Ullah et al. (2024). LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet?). arXiv:2312.12575.

[2] Tony et al. (2024). Prompting Techniques for Secure Code Generation: A Systematic Investigation. arXiv:2407.07064.

[3] OWASP (2021). Threat Modeling Cheat Sheet. cheatsheetseries.owasp.org

[4] NIST (2022). Secure Software Development Framework (SSDF), SP 800-218. doi.org/10.6028/NIST.SP.800-218

[5] OpenSSF Best Practices Working Group (2025). Security-Focused Guide for AI Code Assistant Instructions.

References (continued)

[6] AppSec Engineer (2025). AI-Generated Code Needs Its Own Secure Coding Guidelines. appsecengineer.com

[7] arXiv 2506.23034 (2025). Guiding AI to Fix Its Own Flaws: An Empirical Study on LLM-Driven Secure Code Generation.

[8] Ioannidis et al. (2025, IEEE-ISTAS). Security Degradation in Iterative AI Code Generation. arXiv:2506.11022.

[9] Soares et al. (2024). Property-Based Testing in Practice. ICSE 2024.